

Coding Homework 1: Breaking SHA-1

CS6260: Applied Cryptography

Due Date: See Canvas

This is an individual assignment. Do not discuss code or strategies with your homework collaboration group.

Objectives: Understanding UF-CMA by attacking a real hash function and integrity scheme.

1. Introduction

The goal of this assignment is to break the security of a deterministic hash-based MAC. Namely, let H be the hash function constructed from the compression hash function h via the Merkle-Damgård transform as per Figure 1. Then, to tag a message M consisting of b -bit blocks under a b -bit key K we compute $\text{tag} \leftarrow H(K \parallel M)$.

Recall: Merkle-Damgård Transform



Figure 1: A visualization of the general Merkle-Damgård transform, where h is a function compression an $(n + b)$ -bit input into an n -bit output, $M[i]$ is the i^{th} b -bit block of the message M , and $\langle m \rangle$ is the b -bit encoding of the message's block count, m .

We have provided you with several Python files (See [Code](#) section) that you will work with in this assignment. In particular, **you will be demonstrating that the MAC described above is not UF-CMA secure by modifying `student.py` to create a forged message and a corresponding tag.** Detailed explanations can be found in the following section.

You will be submitting your deliverables via Gradescope.

2. Attack

2.1 Scheme

The MAC prepends a secret symmetric key to an input message, then passes it to the SHA1 algorithm:

$$\text{tag} = \text{SHA1}(\text{password} \parallel \text{message})$$

You don't know the symmetric key, but you do know what length it might be (See the [Objective](#) section)

2.2 Length Extension Attack

A length extension attack is a type of attack where an attacker can use the hash of a message and the length of the message to calculate the hash of that message prepended to another message, where the second message is chosen by the attacker. This can be done without knowing the content of the first message.

This MAC is vulnerable to a **length extension attack**. Specifically, it's possible to forge a valid (message, tag) pair where the message has some arbitrary data appended to the end, while only knowing the **length** of the password (not its actual value).

This is problematic in the MAC construction above because an attacker can include extra information at the end of the message and produce a valid hash without knowing the secret.

2.3 Resources

Apart from the course material, the only resource you may use for this assignment is the [RFC](#) for the SHA1 algorithm that describes how SHA-1 works. You must pay special attention to Sections 3–5 and 6.1. You will also need to carefully read over the code files we have provided you.

3. Code

This section describes the code you have been provided and what you need to modify.

3.1 Files

You've been provided the following library:

- [student.py](#) contains your exploit code that will successfully create a forged message that passes the MAC without knowing the secret key.

This is the only file you will modify for submission.

- `crypto.py` contains a custom SHA1 implementation modeled after Python's `hashlib` module, but is extended with extra parameters that may help with length extension attacks. You will likely need to refer to the RFC to understand how to use them.
- `oracle.py` simulates a UF-CMA adversarial scenario by including the necessary oracles.
- `grader.py` runs your exploit and lets you know whether or not you've successfully forged a message.
- `secret.txt` is an auto-generated secret key used by the oracle when you run the tests. Note that the Gradescope autograder will obviously use different keys.

The docstrings in each file provide further details about these modules; read them!

3.2 Running the Code

You can install the latest version of Python 3 for your system from [here](#); there are no extra dependencies to install. Older versions of Python 3 *may* work, but we cannot make guarantees. To run the local auto-grader, simply execute:

```
python grader.py [your GT username]
```

3.3 Objective

You should make `student.main()` return a (message, tag) pair that:

- includes at least the **original message** and your **GT username** in the forged message,
- hasn't been submitted to the oracle, and
- passes the MAC.

(where message is a sequence of bytes and tag is a hexadecimal string value)

There are two parts to this:

1. You should be able to do this for a **fixed key length** of 64 bytes. (15 points)
2. You should then extend your solution to work for a variable, **arbitrary key length** of up to 100 bytes. (5 points)

4. Submission

You will need to submit the following deliverables via Gradescope. **There are different assignments for each**, so please be careful to submit to the right one!

- [student.py](#) (20 points): Complete `main()` to forge messages via a length extension attack. Remember that the messages you forge should contain at least your GT username and the original message.

You **must** keep the existing structure: nothing should run if you execute `python student.py` on its own, the input parameters should stay the same, and the return value(s) should match the expected format and types.

Submit this to **Homework 4 (Code)** on Gradescope.

The autograder will run a suite of tests to determine your score, offering small suggestions for common mistakes if it encounters them or exception logs if your code doesn't run.

- [report.pdf](#) (5 points): **Briefly** discuss the exploit details. You should touch on, at minimum: how reliable your attack is, its (rough) run-time complexity, the root causes of the vulnerability in the integrity scheme at a high level, and how it could be alleviated (be sure to explain *why* these fixes will remedy the problem.).

Submit this to **Homework 4 (Report)** on Gradescope.

All of these can be answered in one page or less; take care to be succinct *and* precise. **Report content past your second page may be ignored.**